

SISTEMA INMOBILIARIO DE GESTIÓN EN JAVA

Desarrollo de una aplicación de consola orientada a objetos para la gestión de inmuebles y su evolución por versiones.

Iriome Naranjo

Índice

1. RESUMEN DEL PROYECTO.....	6
2. INTRODUCCIÓN.....	7
3. JUSTIFICACIÓN DEL PROYECTO.....	8
4. OBJETIVOS.....	10
5. ALCANCE DEL PROYECTO.....	11
6. ANÁLISIS FUNCIONAL DEL SISTEMA.....	13
1. Registrar vivienda.....	13
2. Listar inventario.....	13
3. Registrar pago.....	13
4. Aplicar IPC.....	14
5. Salir del sistema.....	14
7. ANÁLISIS DE REQUISITOS.....	16
Requisitos funcionales implementados.....	16
8. DISEÑO DEL SISTEMA.....	18
1. Robustez antes que complejidad.....	19
2. Escalabilidad antes que sobrecarga.....	19
3. Coherencia del modelo.....	19
9. ARQUITECTURA ORIENTADA A OBJETOS.....	20
Vivienda → clase base o superclase.....	20
10. DESCRIPCIÓN DETALLADA DE LAS CLASES.....	22
10.1.1 Responsabilidad de la clase.....	22
10.1.2 Atributos de la clase Vivienda.....	22
10.1.3 Constructor de la clase.....	22
10.1.4 Métodos principales de la clase Vivienda.....	23
Métodos de acceso y modificación:.....	23

Métodos de lógica financiera:.....	23
10.1.5 Decisiones de diseño relevantes en Vivienda.....	23
10.2.1 Atributos específicos de Piso.....	24
10.2.2 Constructor y métodos.....	24
10.2.3 Justificación de su existencia como subclase.....	24
10.3.1 Atributo específico de Casa.....	25
10.3.2 Constructor y métodos.....	25
10.3.3 Justificación de su existencia como subclase.....	25
11. GESTIÓN DE MEMORIA Y ALMACENAMIENTO EN EJECUCIÓN.....	25
12. POLIMORFISMO Y TRATAMIENTO DE OBJETOS.....	26
13. DISEÑO DE LA INTERACCIÓN POR CONSOLA.....	27
14. VALIDACIÓN Y ROBUSTEZ DEL SISTEMA.....	29
15. LÓGICA DE NEGOCIO IMPLEMENTADA.....	30
16. DECISIONES DE DISEÑO Y CRITERIO TÉCNICO.....	31
17. LIMITACIONES ACTUALES DEL SISTEMA.....	32
18. VALOR TÉCNICO DE LA IMPLEMENTACIÓN ACTUAL.....	34
19. CASOS DE USO DEL SISTEMA.....	35
Actor principal.....	35
Nombre del caso de uso.....	35
Actor.....	35
Descripción.....	35
Flujo principal.....	35
Resultado esperado.....	36
Observaciones.....	36
Nombre del caso de uso.....	36
Actor.....	36
Descripción.....	36

Flujo principal.....	36
Resultado esperado.....	36
Observaciones.....	36
Nombre del caso de uso.....	37
Actor.....	37
Descripción.....	37
Flujo principal.....	37
Resultado esperado.....	37
Observaciones.....	37
Nombre del caso de uso.....	37
Actor.....	37
Descripción.....	37
Flujo principal.....	37
Resultado esperado.....	38
Observaciones.....	38
20. PRUEBAS REALIZADAS.....	39
Aspectos verificados:.....	39
Resultado obtenido:.....	39
Aspectos verificados:.....	39
Resultado obtenido:.....	39
Aspectos verificados:.....	40
Resultado obtenido:.....	40
Aspectos verificados:.....	40
Resultado obtenido:.....	40
Situaciones probadas:.....	40
Resultado obtenido:.....	40
21. CONTROL DE ERRORES Y ROBUSTEZ.....	41

22. LIMITACIONES ACTUALES DE LA BETA.....	42
23. VISIÓN EVOLUTIVA DEL PROYECTO.....	43
24. ROADMAP DEL PROYECTO.....	44
Elementos incluidos en esta fase:.....	44
Objetivo de esta fase:.....	44
Mejoras previstas:.....	44
Objetivo de esta fase:.....	44
Mejoras previstas:.....	44
Objetivo de esta fase:.....	44
Funcionalidades previstas:.....	45
Objetivo de esta fase:.....	45
25. ADAPTACIÓN AL CONTEXTO CANARIO.....	46
26. POSIBLES MEJORAS FUTURAS.....	47
27. CONCLUSIONES.....	48
28. CIERRE DEL DOCUMENTO.....	49

1. RESUMEN DEL PROYECTO

El presente proyecto consiste en el desarrollo de un **sistema inmobiliario de gestión en Java**, diseñado como una aplicación de consola orientada a objetos para la administración básica de inmuebles en alquiler.

El objetivo principal del sistema es permitir la gestión estructurada de distintos tipos de viviendas, facilitando su registro, clasificación, almacenamiento y tratamiento económico básico dentro de un entorno controlado y robusto.

La aplicación ha sido desarrollada utilizando el lenguaje **Java** en el entorno **BlueJ**, haciendo uso de conceptos fundamentales de **Programación Orientada a Objetos (POO)** como la **herencia**, la **especialización de clases**, la **abstracción del dominio** y el uso de **estructuras dinámicas en memoria** mediante `ArrayList`.

En su estado actual, el sistema permite registrar inmuebles de tipo **Piso** y **Casa**, almacenarlos en memoria, listarlos dinámicamente y aplicar una primera capa de lógica de negocio vinculada a la gestión económica del alquiler.

Entre las funcionalidades ya implementadas destacan:

- el registro de viviendas,
- la diferenciación entre tipos de inmueble,
- la validación robusta de entradas,
- el almacenamiento dinámico en memoria,
- el registro de pagos,
- el cálculo de mensualidades con impuestos,
- la detección de deuda pendiente,
- la proyección de ingresos,
- y la actualización anual del precio base mediante incremento porcentual.

Uno de los aspectos diferenciales del proyecto es que ha sido diseñado inicialmente con un enfoque adaptado al contexto fiscal de **Canarias**, incorporando desde la clase base de vivienda el cálculo del **IGIC** como impuesto aplicable, en lugar del IVA peninsular.

El sistema se encuentra actualmente en fase **BETA**, lo que implica que ya dispone de una base técnica y funcional sólida, aunque aún se reserva parte de su crecimiento para versiones posteriores. Entre sus futuras líneas de evolución se contemplan la mejora de la gestión de usuarios, la persistencia de datos, la expansión territorial y la diferenciación entre distintos perfiles de uso dentro del sistema.

En conjunto, este proyecto no solo representa una solución funcional básica dentro del ámbito inmobiliario, sino también un ejercicio completo de diseño, organización, validación y planificación evolutiva del software.

2. INTRODUCCIÓN

La gestión inmobiliaria, incluso en entornos relativamente pequeños, implica una cantidad considerable de información relacionada con propiedades, precios, pagos, actualizaciones económicas y control administrativo. En muchos casos, estos procesos siguen realizándose mediante métodos poco estructurados, como anotaciones manuales, hojas de cálculo o sistemas fragmentados, lo que aumenta el riesgo de errores, pérdida de información y falta de control sobre los datos.

A partir de esta necesidad, surge la idea de desarrollar un sistema software capaz de ofrecer una base organizada para la gestión de inmuebles, orientado inicialmente a un entorno académico, pero concebido con una visión de evolución progresiva hacia una herramienta más completa.

El presente proyecto se centra en el desarrollo de una aplicación en Java que permita gestionar distintos tipos de viviendas dentro de un sistema inmobiliario básico. La intención no ha sido únicamente construir un programa que “funcione”, sino diseñar una estructura de software bien planteada, capaz de crecer por fases y de incorporar nuevas funcionalidades sin comprometer la base ya construida.

Desde el punto de vista técnico, el proyecto busca representar de forma ordenada el dominio inmobiliario mediante una arquitectura orientada a objetos, utilizando clases, herencia, almacenamiento dinámico y validaciones robustas de entrada.

Desde el punto de vista funcional, el sistema se plantea como una herramienta que permita:

- registrar propiedades,
- diferenciar tipos de vivienda,
- almacenar inmuebles,
- listar información,
- controlar pagos,
- aplicar actualizaciones económicas,
- y sentar las bases para una futura gestión más completa del alquiler y la explotación de inmuebles.

Un aspecto especialmente importante del proyecto es que no ha sido concebido como un producto cerrado desde el principio, sino como un sistema evolutivo desarrollado por etapas. Esta decisión metodológica responde a una lógica clara: construir primero una base robusta y defendible, para posteriormente ampliar sus capacidades de forma controlada y coherente.

Por ello, el proyecto se estructura en distintas fases de desarrollo, comenzando por una **BETA funcional**, a partir de la cual se proyectan futuras versiones más completas como la **V1.0**, la **V1.1**, la **V2.0** y la **V3.0**.

En consecuencia, este trabajo debe entenderse no solo como una implementación técnica puntual, sino como la primera etapa de una solución software pensada para evolucionar de manera ordenada.

3. JUSTIFICACIÓN DEL PROYECTO

La elección de este proyecto responde a una combinación de razones funcionales, técnicas, académicas y de diseño de software.

Desde el punto de vista funcional, la gestión de inmuebles constituye un escenario muy adecuado para el desarrollo de una aplicación de administración estructurada, ya que implica trabajar con datos organizados, distintos tipos de entidades, relaciones lógicas y operaciones económicas recurrentes.

A diferencia de ejercicios puramente abstractos, el ámbito inmobiliario permite modelar situaciones reales y fácilmente reconocibles, como por ejemplo:

- registrar una propiedad,
- asignarle un precio,
- diferenciar su tipo,
- calcular su mensualidad,
- registrar pagos,
- o aplicar actualizaciones de precio.

Esto convierte el proyecto en un contexto muy apropiado para trasladar el mundo real a una estructura software bien organizada.

Desde el punto de vista técnico, el proyecto ofrece un entorno muy útil para aplicar y consolidar conceptos fundamentales de programación, entre ellos:

- **Programación Orientada a Objetos,**
- **abstracción,**
- **encapsulamiento,**
- **herencia,**
- **polimorfismo,**
- **gestión de estructuras dinámicas,**
- **y validación robusta de datos.**

Además, este proyecto permite ir más allá del simple desarrollo de funciones aisladas, ya que obliga a pensar el sistema como un conjunto coherente de decisiones de diseño. En otras palabras, no se trata únicamente de “programar opciones de menú”, sino de plantear:

- cómo se organiza el modelo de clases,
- qué responsabilidades debe tener cada objeto,
- qué lógica debe vivir en cada nivel del sistema,
- cómo proteger la aplicación frente a errores,
- y cómo permitir su crecimiento sin romper la base existente.

Desde una perspectiva académica, esto aporta mucho valor al proyecto, ya que demuestra no solo capacidad de codificación, sino también criterio en el diseño y la planificación de software.

Otro elemento que justifica especialmente este desarrollo es su enfoque progresivo. En lugar de intentar construir desde el principio una aplicación demasiado ambiciosa o

sobrecargada, se ha optado por desarrollar primero una **BETA funcional y robusta**, sobre la cual se apoyará la evolución futura del sistema.

Esta decisión es importante porque refleja una forma de trabajar más cercana al desarrollo real de software: primero se construye una base estable, después se consolida y finalmente se amplía.

Por último, el proyecto incorpora una dimensión especialmente interesante al estar inicialmente adaptado al contexto de **Canarias**, utilizando el **IGIC** como referencia fiscal en la lógica económica del sistema. Esto le aporta realismo, especificidad y un enfoque de producto más definido, diferenciándolo de una solución completamente genérica desde sus primeras fases.

En conjunto, este proyecto se justifica tanto por su utilidad práctica como por su valor formativo, técnico y metodológico.

4. OBJETIVOS

4.1 Objetivo general

Desarrollar una aplicación de consola en Java orientada a la gestión inmobiliaria básica, capaz de registrar, almacenar, clasificar y administrar inmuebles en alquiler, incorporando una arquitectura orientada a objetos robusta, lógica económica inicial y una base escalable para futuras versiones del sistema.

4.2 Objetivos específicos

- Diseñar una estructura de clases coherente para representar distintos tipos de viviendas.
- Implementar un sistema de registro de inmuebles por consola.
- Diferenciar el tratamiento de **Piso** y **Casa** dentro del sistema.
- Almacenar dinámicamente las viviendas registradas en memoria.
- Mostrar listados organizados de inmuebles registrados.
- Permitir el registro de pagos asociados a las viviendas.
- Calcular la mensualidad total aplicando la fiscalidad correspondiente.
- Detectar pagos pendientes y controlar el estado de abono de cada inmueble.
- Permitir la actualización porcentual del precio base de las propiedades.
- Preparar el sistema para futuras ampliaciones funcionales.

4.3 Objetivos técnicos

- Aplicar correctamente principios de **Programación Orientada a Objetos**.
- Utilizar **herencia** para representar la jerarquía entre tipos de vivienda.
- Gestionar distintos objetos dentro de una misma colección mediante `ArrayList<Vivienda>`.
- Implementar una estructura de menú por consola organizada y funcional.
- Aplicar validación robusta mediante `try-catch`, `while` y control de opciones.
- Incorporar una primera capa de lógica de negocio dentro del modelo de clases.
- Diseñar el sistema de forma mantenible y escalable.

4.4 Objetivos de aprendizaje

- Consolidar el uso práctico de clases, objetos y jerarquías de herencia.
- Aprender a estructurar un proyecto real más allá de ejercicios simples.
- Comprender cómo separar datos, comportamiento y lógica de interacción.
- Desarrollar criterio técnico para decidir qué implementar en cada fase.
- Mejorar la capacidad de documentar, justificar y defender decisiones de diseño.

5. ALCANCE DEL PROYECTO

El alcance del presente proyecto se centra en el desarrollo de una **BETA funcional** de un sistema de gestión inmobiliaria en Java, ejecutado por consola y orientado a la administración básica de viviendas en alquiler.

Esta fase ha sido diseñada para cubrir la base técnica, estructural y funcional del sistema, dejando conscientemente fuera determinadas capas más avanzadas que se consideran propias de futuras versiones.

La delimitación del alcance no responde a una falta de ambición del proyecto, sino a una estrategia de desarrollo progresivo, en la que se prioriza primero la estabilidad, la claridad arquitectónica y la robustez del núcleo del software.

5.1 Funcionalidades incluidas en la BETA

La versión actual del sistema permite:

- registrar nuevas viviendas,
- diferenciar entre inmuebles de tipo **Piso** y **Casa**,
- solicitar datos específicos según el tipo de propiedad,
- almacenar las viviendas en memoria,
- listar el inventario de inmuebles registrados,
- registrar pagos sobre viviendas concretas,
- comprobar si una mensualidad está pagada por completo,
- calcular el importe pendiente de pago,
- calcular la mensualidad total con impuestos,
- aplicar un incremento porcentual a todas las propiedades,
- y mantener una interacción guiada mediante menú por consola.

5.2 Elementos ya implementados en la lógica del sistema

A diferencia de una fase puramente estructural, la BETA actual ya incorpora una primera capa de **lógica de negocio real**.

Esta lógica incluye:

- cálculo del precio final con **IGIC**,
- control del total pagado por vivienda,
- cálculo del saldo pendiente,
- validación de pago completo,
- proyección de ingresos a varios meses,
- y revisión del precio base mediante subida porcentual anual.

Esto significa que el sistema ya no se limita a registrar propiedades, sino que empieza a comportarse como una herramienta de gestión con utilidad operativa básica.

5.3 Funcionalidades fuera del alcance actual

En la fase BETA quedan fuera del alcance actual del proyecto los siguientes elementos:

- interfaz gráfica,
- base de datos,
- persistencia permanente,
- autenticación de usuarios,
- historial avanzado de operaciones,
- sistema multiusuario,
- estadísticas complejas,
- informes financieros detallados,
- soporte fiscal completo para Península,
- automatización documental,
- y explotación avanzada de inmuebles turísticos.

Estas funciones no se consideran descartadas, sino planificadas para fases posteriores del roadmap del sistema.

6. ANÁLISIS FUNCIONAL DEL SISTEMA

6.1 Finalidad funcional del programa

Desde un punto de vista funcional, el sistema ha sido diseñado para permitir una gestión básica pero estructurada de inmuebles en alquiler, proporcionando al usuario una herramienta capaz de registrar, consultar y administrar propiedades dentro de un entorno controlado.

El programa actúa actualmente como un núcleo administrativo sencillo, orientado principalmente a la organización interna de viviendas y a la gestión económica inicial de sus mensualidades.

6.2 Funciones principales disponibles

En su estado actual, el sistema permite al usuario realizar las siguientes operaciones principales:

1. Registrar vivienda

El sistema permite registrar un nuevo inmueble introduciendo sus datos principales, como la dirección y el precio base, y posteriormente especificar si se trata de un **Piso** o una **Casa**.

En función de la elección realizada, el programa solicita información adicional específica:

- en caso de **Piso**, se solicita la **planta** y la **puerta**;
- en caso de **Casa**, se solicita la **superficie de parcela**.

Una vez introducidos y validados los datos, el sistema crea el objeto correspondiente y lo almacena en memoria.

2. Listar inventario

El sistema permite visualizar el conjunto de viviendas registradas en memoria.

Durante este proceso, el programa recorre dinámicamente la colección de inmuebles y muestra por pantalla la información general y específica de cada propiedad.

Esta funcionalidad utiliza la jerarquía orientada a objetos del sistema para distinguir correctamente qué tipo de datos mostrar según si el inmueble es un **Piso** o una **Casa**.

3. Registrar pago

El sistema permite registrar un pago sobre una vivienda concreta.

Para ello:

- filtra las propiedades que aún tienen importe pendiente,
- las muestra numeradas al usuario,
- permite seleccionar una de ellas,
- y solicita la cantidad que se desea abonar.

Una vez realizado el pago, el programa actualiza el estado económico del inmueble y comprueba si la mensualidad ha sido completamente cubierta o si aún existe deuda pendiente.

Esta funcionalidad representa una de las partes más relevantes de la lógica de negocio actual del proyecto.

4. Aplicar IPC

El sistema permite aplicar una subida porcentual sobre el precio base de todas las viviendas registradas.

Esta operación recorre el conjunto de inmuebles almacenados y actualiza su precio base, recalculando automáticamente la mensualidad final con impuestos.

Desde el punto de vista funcional, esta característica simula una actualización masiva de precios, muy coherente con el ámbito administrativo e inmobiliario del sistema.

5. Salir del sistema

El usuario puede cerrar la aplicación desde el menú principal, finalizando la ejecución del programa de forma controlada.

6.3 Flujo funcional general

El funcionamiento del sistema sigue un flujo secuencial basado en menú.

De forma general, el comportamiento del programa puede describirse de la siguiente manera:

1. El usuario inicia la aplicación.
2. El sistema muestra el menú principal.
3. El usuario selecciona una opción.
4. El programa solicita la información necesaria.
5. Se validan las entradas.
6. Se ejecuta la acción correspondiente.
7. Se actualiza el estado del sistema.
8. El usuario vuelve al menú principal o finaliza la ejecución.

Este flujo resulta adecuado para una aplicación de consola centrada en gestión, ya que ofrece una navegación clara, predecible y fácilmente controlable.

6.4 Enfoque práctico del sistema

Aunque el proyecto se encuentra aún en una fase BETA, su orientación funcional ya refleja el comportamiento básico de una herramienta administrativa inmobiliaria.

En términos prácticos, el sistema podría servir como base para:

- pequeñas agencias inmobiliarias,
- propietarios con varias viviendas,
- entornos de alquiler sencillos,
- o simulaciones académicas de gestión patrimonial.

Por tanto, el software ya posee una utilidad funcional reconocible, aunque todavía no incorpore todas las capas que tendría una solución plenamente profesional.

7. ANÁLISIS DE REQUISITOS

7.1 Requisitos funcionales

Los requisitos funcionales describen las operaciones que el sistema debe ser capaz de realizar.

Requisitos funcionales implementados

- El sistema debe permitir registrar nuevas viviendas.
- El sistema debe permitir diferenciar entre inmuebles de tipo **Piso** y **Casa**.
- El sistema debe solicitar atributos específicos según el tipo de vivienda.
- El sistema debe almacenar los inmuebles registrados en memoria.
- El sistema debe permitir listar el inventario actual de propiedades.
- El sistema debe permitir registrar pagos sobre viviendas concretas.
- El sistema debe calcular el precio final mensual aplicando **IGIC**.
- El sistema debe calcular el importe pendiente de pago.
- El sistema debe determinar si una mensualidad ha sido abonada por completo.
 - El sistema debe permitir aplicar un incremento porcentual sobre el precio base.
 - El sistema debe recalcular automáticamente la mensualidad tras la actualización del precio.

7.2 Requisitos no funcionales

- El sistema debe ser **estable** durante la ejecución.
- El sistema debe ser **resistente a errores de entrada del usuario**.
- El sistema debe ser **fácil de usar desde consola**.
- El sistema debe ser **comprensible** para usuarios no técnicos.
- El sistema debe ser **mantenible**.
- El sistema debe ser **escalable** para futuras versiones.
- El sistema debe ser **académicamente defendible**.
- El sistema debe mantener una estructura clara y ordenada.

7.3 Requisitos técnicos

- Lenguaje de programación: **Java**
- Entorno de desarrollo: **BlueJ**
- Interfaz de interacción: **consola**
- Gestión de datos: `ArrayList<Vivienda>`
- Entrada de usuario: `Scanner`
- Modelo estructural: **Programación Orientada a Objetos**
- Validación: `try-catch`, `while`, control de opciones y control de tipo de dato

7.4 Requisitos futuros previstos

Aunque no forman parte del alcance actual, el sistema ha sido concebido para soportar en el futuro requisitos como:

- persistencia de datos,
- gestión de usuarios,
- historial de operaciones,
- distintos perfiles de uso,
- adaptación fiscal peninsular,
- soporte para más tipos de inmuebles,
- y una futura capa de interfaz más avanzada.

Estos requisitos se desarrollarán de forma progresiva dentro de la hoja de ruta del proyecto.

8. DISEÑO DEL SISTEMA

8.1 Diseño general de la solución

El sistema ha sido diseñado como una aplicación de consola en Java orientada a la gestión básica de inmuebles en alquiler. Desde el punto de vista estructural, se ha optado por un diseño basado en **Programación Orientada a Objetos (POO)**, ya que este paradigma permite representar de forma más natural y escalable los distintos elementos del dominio inmobiliario.

La decisión de utilizar un diseño orientado a objetos responde a la necesidad de organizar el proyecto de forma clara, separando:

- la representación de los inmuebles,
- la lógica económica asociada a cada vivienda,
- y el flujo de interacción con el usuario.

En lugar de construir todo el sistema dentro de una única clase o mediante variables sueltas, se ha planteado una arquitectura en la que cada tipo de entidad tenga una responsabilidad bien definida.

De este modo, el proyecto se apoya sobre tres clases principales:

- **Vivienda**
- **Piso**
- **Casa**

y una clase de ejecución principal:

- **GestionInmobiliaria**

Esta separación permite que el sistema sea más:

- entendible,
- mantenible,
- modular,
- reutilizable,
- y ampliable.

Además, el diseño actual no solo permite registrar y listar inmuebles, sino que también integra una primera capa de lógica de negocio dentro de los propios objetos, lo cual constituye una decisión de diseño importante y adecuada para este tipo de aplicación.

8.2 Filosofía de diseño aplicada

El desarrollo del sistema ha seguido una filosofía de construcción progresiva, basada en tres principios principales:

1. Robustez antes que complejidad

Se ha priorizado que el sistema sea estable y resistente a errores de entrada antes de incorporar funcionalidades más avanzadas.

2. Escalabilidad antes que sobrecarga

En lugar de intentar construir desde el inicio un sistema demasiado grande o complejo, se ha preferido crear una base sólida que permita crecer por fases.

3. Coherencia del modelo

Cada clase ha sido diseñada para representar de forma clara una parte del dominio real, evitando mezclar demasiadas responsabilidades en un único bloque de código.

Esta filosofía encaja especialmente bien con una fase **BETA**, donde el objetivo principal no es tener un producto final cerrado, sino construir correctamente los cimientos del sistema.

9. ARQUITECTURA ORIENTADA A OBJETOS

9.1 Justificación del uso de Programación Orientada a Objetos

La **Programación Orientada a Objetos** ha sido elegida como base del proyecto porque permite modelar de forma directa los elementos del ámbito inmobiliario mediante entidades que poseen:

- atributos,
- comportamiento,
- relaciones,
- y especializaciones.

En este contexto, una vivienda no se trata como un simple conjunto de variables, sino como un **objeto** con identidad propia y con capacidad para realizar operaciones relevantes dentro del sistema.

Esta aproximación aporta varias ventajas importantes:

- mejora la organización del código,
- facilita la reutilización,
- reduce la repetición,
- y permite ampliar el sistema con mayor facilidad.

Además, la POO resulta especialmente adecuada en este proyecto porque existen elementos que comparten características comunes, pero que también necesitan comportamientos o atributos específicos.

Ese es precisamente el caso de los inmuebles **Piso** y **Casa**, que comparten una base común, pero requieren información adicional distinta.

9.2 Estructura jerárquica de clases

El sistema ha sido diseñado siguiendo una estructura jerárquica basada en **herencia**, donde existe una clase general que representa el concepto abstracto de vivienda, y varias clases hijas que especializan dicho concepto.

La jerarquía actual del sistema es la siguiente:

- **Vivienda** → **clase base o superclase**
 - **Piso** → subclase especializada
 - **Casa** → subclase especializada

Este diseño permite que los atributos y métodos comunes se centralicen en la clase principal, mientras que cada subtipo añade únicamente la información que le corresponde.

Desde un punto de vista conceptual, esta decisión evita duplicar código y mejora notablemente la mantenibilidad del proyecto.

9.3 Ventajas del modelo adoptado

La arquitectura utilizada aporta varias ventajas relevantes:

- evita la duplicación de atributos comunes,
- permite tratar distintos tipos de inmuebles bajo una misma referencia,
- facilita la ampliación futura del sistema,
- mejora la legibilidad del código,
- y permite aplicar polimorfismo en la gestión de colecciones.

Gracias a ello, el sistema puede almacenar diferentes tipos de vivienda dentro de una misma estructura dinámica y seguir tratándolos de forma organizada.

10. DESCRIPCIÓN DETALLADA DE LAS CLASES

10.1 Clase Vivienda

La clase **Vivienda** constituye la base del sistema y representa el concepto general de inmueble en alquiler dentro de la aplicación.

Su función principal es centralizar toda la información común a cualquier tipo de vivienda, así como parte de la lógica económica asociada a su gestión.

Se trata, por tanto, de la clase más importante del modelo actual.

10.1.1 Responsabilidad de la clase

La clase **Vivienda** ha sido diseñada para asumir tres responsabilidades principales:

- representar la información básica de cualquier inmueble,
- gestionar su cálculo económico mensual,
- y controlar su estado financiero básico.

Esto la convierte en una clase de modelo con capacidad operativa, y no únicamente en un contenedor de datos.

10.1.2 Atributos de la clase Vivienda

La clase incorpora los siguientes atributos:

- **IGIC** → constante estática que representa el impuesto aplicable en Canarias (7.0%)
- **direccion** → dirección física de la vivienda
- **precioBase** → importe mensual del alquiler sin impuestos
- **mensualidadTotal** → importe final mensual tras aplicar el IGIC
- **totalPagadoMes** → cantidad total abonada hasta el momento durante el mes actual

Estos atributos permiten representar tanto el estado descriptivo como el estado económico de una vivienda dentro del sistema.

10.1.3 Constructor de la clase

La clase dispone de un constructor principal que recibe:

- la dirección de la vivienda
- el precio base del alquiler

A partir de estos datos, el sistema inicializa automáticamente:

- la mensualidad total con impuestos
- y el total pagado del mes en cero

Esta decisión mejora la consistencia interna del objeto, ya que evita que una vivienda se cree en un estado incompleto o incoherente.

10.1.4 Métodos principales de la clase **Vivienda**

La clase incluye diversos métodos que cubren tanto acceso a datos como lógica económica.

Métodos de acceso y modificación:

- `getDireccion()`
- `getPrecioBase()`
- `setDireccion()`
- `setPrecioBase()`

Estos métodos permiten consultar y actualizar los atributos principales del inmueble respetando el principio de encapsulamiento.

Métodos de lógica financiera:

- `getPrecioConImpuestos()`
- `registrarPago(double cuota)`
- `getPendienteDePago()`
- `isPagadoCompleto()`
- `getProyeccionIngresos(int meses)`
- `aplicarSubidaAnual(double porcentaje)`

Estos métodos representan una parte fundamental de la lógica del sistema.

Su inclusión en la clase **Vivienda** está justificada porque se trata de operaciones que cualquier inmueble del sistema debe poder realizar, independientemente de si se trata de un piso o de una casa.

10.1.5 Decisiones de diseño relevantes en **Vivienda**

Una de las decisiones más acertadas del diseño actual es haber concentrado la lógica económica común dentro de la clase base.

Esto evita repetir métodos como:

- cálculo de impuestos,
- gestión de pagos,
- deuda pendiente,
- o revisión del precio

en cada subtipo de vivienda.

Además, permite que cualquier futura subclase (por ejemplo, un **Garaje**, un **Local Comercial** o un **Apartamento Turístico**) pueda heredar automáticamente estas capacidades sin necesidad de reimplementarlas.

10.2 Clase Piso

La clase **Piso** representa un tipo específico de vivienda ubicado dentro de una estructura de propiedad horizontal, como puede ser un edificio residencial.

Esta clase hereda de **Vivienda** toda la información común y añade únicamente los atributos específicos necesarios para identificar correctamente este tipo de inmueble.

10.2.1 Atributos específicos de Piso

La clase incorpora los siguientes atributos propios:

- **planta** → número de planta donde se encuentra el inmueble
- **puerta** → identificador de la puerta o unidad concreta

Estos datos resultan necesarios para distinguir adecuadamente un piso dentro de un edificio.

10.2.2 Constructor y métodos

El constructor de **Piso** recibe:

- dirección
- precio base
- planta
- puerta

Los dos primeros datos se delegan al constructor de la superclase mediante `super(...)`, mientras que los atributos específicos se inicializan dentro de la propia subclase.

La clase también incorpora sus correspondientes métodos:

- `getPlanta()`
- `getPuerta()`
- `setPlanta()`
- `setPuerta()`

10.2.3 Justificación de su existencia como subclase

La creación de una subclase específica para **Piso** está plenamente justificada, ya que este tipo de inmueble necesita información adicional que no tendría sentido incluir en todas las viviendas del sistema.

Si atributos como **planta** o **puerta** se hubieran colocado en la clase base **Vivienda**, se estaría forzando a que todas las viviendas del sistema los tuviesen, incluso aquellas en las que no resultan aplicables, como una casa independiente.

Por ello, la especialización mediante herencia constituye la solución más limpia y correcta.

10.3 Clase Casa

La clase **Casa** representa una vivienda de tipo unifamiliar o independiente.

Al igual que **Piso**, hereda de la clase **Vivienda** toda la información general y añade únicamente un atributo específico relacionado con el terreno asociado al inmueble.

10.3.1 Atributo específico de Casa

La clase incorpora el siguiente atributo:

- **metrosParcela** → superficie del terreno o parcela asociada a la vivienda

Este atributo resulta coherente con el tipo de inmueble representado, ya que una casa puede disponer de terreno privado, a diferencia de un piso.

10.3.2 Constructor y métodos

El constructor de **Casa** recibe:

- dirección
- precio base
- metros de parcela

Los dos primeros valores se inicializan mediante el constructor heredado de **Vivienda**, mientras que el atributo específico se gestiona dentro de la propia subclase.

La clase incorpora además:

- `getMetrosParcela()`
- `setMetrosParcela()`

10.3.3 Justificación de su existencia como subclase

La clase **Casa** permite representar un tipo de vivienda que, aunque comparte la base económica y administrativa del sistema, requiere un dato físico específico distinto al de otras tipologías.

Esta especialización mejora la fidelidad del modelo respecto al dominio real y refuerza la corrección del diseño orientado a objetos.

11. GESTIÓN DE MEMORIA Y ALMACENAMIENTO EN EJECUCIÓN

11.1 Uso de ArrayList<Vivienda>

Para almacenar los inmuebles registrados durante la ejecución del programa, se ha utilizado una estructura dinámica del tipo:

`ArrayList<Vivienda>`

Esta decisión resulta especialmente adecuada porque permite guardar en una misma colección objetos de distintas subclases siempre que hereden de la clase base **Vivienda**.

Gracias a ello, el sistema puede almacenar tanto objetos de tipo **Piso** como de tipo **Casa** dentro de una misma lista.

11.2 Justificación del uso de una colección dinámica

Se ha elegido ArrayList en lugar de un array tradicional por varias razones:

- no requiere un tamaño fijo inicial,
- permite añadir elementos de forma flexible,
- simplifica la gestión del inventario,
- y resulta muy adecuada para una fase BETA sin persistencia en base de datos.

Además, el uso de una colección dinámica facilita enormemente futuras ampliaciones del sistema.

11.3 Implicaciones del diseño en memoria

Actualmente, todos los datos del sistema existen únicamente mientras el programa está en ejecución.

Esto significa que:

- las viviendas registradas se almacenan temporalmente,
- y se pierden al cerrar la aplicación.

Esta limitación es completamente coherente con el alcance de la versión BETA y se considera una fase previa natural antes de incorporar persistencia permanente en futuras versiones.

12. POLIMORFISMO Y TRATAMIENTO DE OBJETOS

12.1 Uso del polimorfismo en el sistema

Uno de los elementos más importantes del diseño actual es el uso de **polimorfismo**, especialmente visible en el manejo de la colección ArrayList<Vivienda>.

Gracias a esta decisión, el programa puede tratar distintos tipos de inmuebles bajo una misma referencia común.

Por ejemplo, tanto un objeto de tipo **Piso** como uno de tipo **Casa** pueden ser almacenados como si fuesen simplemente una **Vivienda**, sin perder por ello su naturaleza real.

Esto aporta flexibilidad y mejora la estructura del programa.

12.2 Uso de instanceof y casting

Para mostrar correctamente la información específica de cada vivienda durante el listado, el sistema utiliza:

- instanceof
- y conversión de tipos (**casting**)

Este enfoque permite comprobar en tiempo de ejecución si una vivienda concreta es un **Piso** o una **Casa**, y a partir de ahí acceder a sus atributos particulares.

Por ejemplo:

- si el objeto es un **Piso**, se muestran su planta y puerta;
- si el objeto es una **Casa**, se muestran los metros de parcela.

12.3 Valor académico y técnico de esta decisión

Aunque en versiones futuras podría evolucionarse hacia soluciones más avanzadas basadas en métodos polimórficos sobrescritos, el uso actual de instanceof resulta completamente válido dentro del contexto de esta fase del proyecto.

Además, demuestra comprensión práctica de:

- herencia,
- jerarquía de tipos,
- tratamiento heterogéneo de objetos,
- y acceso seguro a subclases.

Por tanto, esta decisión es técnicamente defendible y adecuada para el nivel actual del sistema.

13. DISEÑO DE LA INTERACCIÓN POR CONSOLA

13.1 Clase principal GestionInmobiliaria

La clase **GestionInmobiliaria** actúa como punto de entrada del programa y contiene el flujo principal de interacción con el usuario.

Su función principal es coordinar la ejecución general del sistema mediante un menú por consola, permitiendo al usuario navegar entre las distintas funcionalidades disponibles.

13.2 Estructura del menú principal

El sistema presenta al usuario un menú con cinco opciones principales:

1. Registrar vivienda
2. Listar inventario

3. Registrar pago
4. Aplicar IPC
5. Salir

Este diseño permite una navegación sencilla, directa y fácilmente comprensible incluso para usuarios sin conocimientos técnicos.

13.3 Flujo de funcionamiento del menú

El menú principal se ejecuta dentro de una estructura repetitiva `do-while`, lo que permite que el sistema permanezca activo hasta que el usuario seleccione la opción de salida.

El flujo de funcionamiento general es el siguiente:

1. Mostrar menú
2. Leer opción
3. Validar entrada
4. Ejecutar opción correspondiente
5. Volver al menú

Este patrón es muy adecuado para aplicaciones de consola de carácter administrativo.

13.4 Uso de `switch` como estructura de control

La lógica de selección del menú ha sido implementada mediante una estructura `switch`, lo que aporta claridad y una organización muy limpia del flujo principal.

Cada opción del sistema se encuentra separada dentro de su propio bloque `case`, lo que facilita:

- la lectura del código,
- el mantenimiento,
- y la incorporación de nuevas funcionalidades en el futuro.

14. VALIDACIÓN Y ROBUSTEZ DEL SISTEMA

14.1 Importancia de la validación en el proyecto

Uno de los objetivos técnicos más importantes del proyecto ha sido garantizar que el sistema sea resistente a errores provocados por entradas incorrectas del usuario.

En aplicaciones de gestión por consola, una gran parte de los fallos potenciales no proviene del cálculo interno, sino del uso incorrecto de la aplicación por parte del usuario final.

Por ello, se ha prestado especial atención a la validación de datos.

14.2 Estrategia de validación implementada

El sistema utiliza una combinación de:

- `try-catch`
- estructuras `while`
- control de rangos válidos
- y vaciado del buffer de entrada

para impedir que el programa falle cuando el usuario introduce datos no válidos.

Este enfoque se aplica en múltiples puntos del sistema, como por ejemplo:

- selección de opciones del menú,
- introducción del precio base,
- elección entre **Piso** o **Casa**,
- planta del piso,
- metros de parcela,
- cantidad de pago,
- y porcentaje de subida anual.

14.3 Ventajas de esta estrategia

La estrategia de validación utilizada aporta varias ventajas importantes:

- evita cierres inesperados del programa,
- mejora la experiencia de uso,
- hace el sistema más robusto,
- y transmite una sensación de control y fiabilidad.

Desde una perspectiva de calidad del software, esta parte constituye uno de los elementos más sólidos del proyecto actual.

15. LÓGICA DE NEGOCIO IMPLEMENTADA

15.1 Introducción a la lógica de negocio del sistema

Aunque el proyecto se encuentra aún en fase BETA, ya incorpora una primera capa funcional de **lógica de negocio**, lo cual representa un avance importante respecto a un simple sistema de registro de datos.

La lógica de negocio puede entenderse como el conjunto de reglas, cálculos y comportamientos que convierten una estructura técnica en una herramienta con utilidad real.

En este proyecto, dicha lógica se ha centralizado principalmente en la clase **Vivienda**.

15.2 Cálculo del precio con IGIC

Cada vivienda incorpora automáticamente el cálculo del precio final mensual a partir del precio base y el impuesto aplicable en Canarias.

Esta operación se realiza mediante el método:

```
getPrecioConImpuestos()
```

La incorporación de esta funcionalidad demuestra que el sistema no solo almacena un valor económico, sino que también es capaz de transformarlo conforme a una regla fiscal concreta.

15.3 Registro de pagos

El sistema permite registrar pagos parciales o completos sobre una vivienda concreta mediante el método:

```
registrarPago(double cuota)
```

Cada vez que se realiza un abono, el importe se acumula dentro del atributo correspondiente al total pagado del mes.

Esto permite mantener un seguimiento interno del estado financiero de cada inmueble.

15.4 Cálculo de deuda pendiente

La aplicación calcula el saldo pendiente de cada vivienda mediante el método:

```
getPendienteDePago()
```

Gracias a ello, el sistema puede informar en todo momento de cuánto falta por abonar para cubrir la mensualidad total.

Esta funcionalidad resulta especialmente útil dentro de una herramienta de gestión de alquileres.

15.5 Verificación de pago completo

El método:

```
isPagadoCompleto()
```

permite determinar si la mensualidad ha sido completamente cubierta o no.

Esto facilita decisiones posteriores dentro del sistema, como por ejemplo:

- mostrar solo viviendas pendientes de pago,
- o informar al usuario del estado de cobro de cada inmueble.

15.6 Proyección de ingresos

La clase **Vivienda** también incorpora una funcionalidad de estimación económica mediante el método:

```
getProyeccionIngresos(int meses)
```

Este método permite calcular cuánto dinero se espera ingresar en un periodo futuro determinado, siempre que este se encuentre dentro de un rango lógico anual.

Aunque actualmente esta funcionalidad aún no está integrada dentro del menú principal, su presencia demuestra que la lógica del modelo ya está pensada para una gestión económica más avanzada.

15.7 Aplicación de subida anual

Otra funcionalidad destacable es la posibilidad de actualizar el precio base de las viviendas mediante una subida porcentual.

Esto se realiza a través del método:

```
aplicarSubidaAnual(double porcentaje)
```

Esta operación simula una revisión anual de precios y representa una función bastante realista dentro del contexto inmobiliario.

Además, el método recalcula automáticamente la mensualidad total con impuestos tras la actualización, lo que mantiene la coherencia económica del objeto.

16. DECISIONES DE DISEÑO Y CRITERIO TÉCNICO

16.1 Separación entre modelo y flujo de interacción

Una de las decisiones más importantes del proyecto ha sido no concentrar toda la lógica dentro del `main`.

En su lugar, se ha optado por una separación razonable entre:

- las clases que representan el dominio del sistema (**Vivienda**, **Piso**, **Casa**)

- y la clase que coordina la interacción con el usuario (**GestionInmobiliaria**)

Esta decisión mejora la organización del proyecto y facilita su evolución futura.

16.2 Inclusión de lógica dentro de los objetos

Otra decisión acertada ha sido introducir parte de la lógica económica directamente dentro de la clase **Vivienda**.

Esto permite que cada objeto “sepa” operar sobre sí mismo, en lugar de obligar a que todos los cálculos se hagan desde fuera.

Este enfoque está mucho más alineado con la filosofía de la **Programación Orientada a Objetos**.

16.3 Diseño preparado para crecimiento

Aunque el sistema actual es todavía una BETA, ya ha sido diseñado de forma que pueda crecer en el futuro sin necesidad de rehacer completamente su base.

Por ejemplo, en futuras versiones será posible añadir:

- nuevos tipos de inmuebles,
- nuevos perfiles de usuario,
- nuevos cálculos económicos,
- persistencia de datos,
- o una interfaz gráfica

sin romper la estructura principal ya implementada.

Esta previsión de crecimiento constituye uno de los puntos fuertes del proyecto.

17. LIMITACIONES ACTUALES DEL SISTEMA

17.1 Limitaciones propias de la fase BETA

Como todo sistema en desarrollo, la versión actual presenta ciertas limitaciones que no deben interpretarse como errores de planteamiento, sino como elementos conscientemente reservados para futuras fases.

Entre las principales limitaciones actuales destacan:

- ausencia de base de datos,
- pérdida de datos al cerrar el programa,
- ausencia de interfaz gráfica,
- inexistencia de usuarios diferenciados,
- y falta de automatización documental.

Estas limitaciones son coherentes con el alcance previsto para esta etapa del proyecto.

17.2 Limitaciones funcionales aún no integradas

Además, aunque parte de la lógica ya existe a nivel de clases, todavía no toda ella está completamente expuesta al usuario final desde el menú principal.

Por ejemplo:

- la proyección de ingresos existe a nivel de objeto,
- pero aún no dispone de una opción específica visible dentro del flujo principal.

Esto no representa una debilidad estructural, sino una funcionalidad parcialmente preparada para futuras iteraciones.

18. VALOR TÉCNICO DE LA IMPLEMENTACIÓN ACTUAL

La implementación actual del proyecto posee un valor técnico notable dentro del contexto de una fase BETA académica y evolutiva.

El sistema no se limita a una simple colección de clases o a un ejercicio básico de consola, sino que ya demuestra integración de varios conceptos fundamentales del desarrollo de software:

- modelado orientado a objetos,
- especialización por herencia,
- almacenamiento dinámico,
- validación robusta,
- gestión económica básica,
- control de pagos,
- y diseño preparado para evolución futura.

Esto convierte la implementación actual en una base sólida, coherente y bien defendible sobre la que continuar el desarrollo del sistema en versiones posteriores.

19. CASOS DE USO DEL SISTEMA

19.1 Introducción a los casos de uso

Los casos de uso permiten describir de forma clara cómo interactúa un usuario con el sistema y qué funcionalidades tiene disponibles dentro de la aplicación.

En el contexto de este proyecto, los casos de uso ayudan a traducir la lógica del programa a una visión más funcional y comprensible, tanto para usuarios finales como para lectores técnicos.

Actualmente, la versión BETA del sistema dispone de un conjunto reducido pero sólido de operaciones principales orientadas a la gestión inmobiliaria básica.

19.2 Actor principal del sistema

En la fase actual del proyecto, el sistema ha sido diseñado para ser utilizado por un único perfil funcional:

Actor principal

Gestor inmobiliario / administrador del sistema

Este actor representa a la persona encargada de registrar viviendas, revisar el inventario, gestionar pagos y aplicar revisiones económicas.

En futuras versiones, el sistema evolucionará para incorporar perfiles diferenciados como:

- usuario arrendatario,
- usuario inquilino,
- y administrador ampliado.

Sin embargo, en la BETA actual, el sistema opera bajo una única capa de uso administrativo.

19.3 Caso de uso: Registrar vivienda

Nombre del caso de uso

Registrar una nueva vivienda

Actor

Gestor inmobiliario

Descripción

Permite introducir una nueva vivienda dentro del sistema, almacenándola en memoria para su posterior gestión.

Flujo principal

1. El usuario accede a la opción “Registrar Vivienda”.

2. El sistema solicita la dirección del inmueble.
3. El sistema solicita el precio base del alquiler.
4. El sistema pregunta si se trata de un piso o una casa.
5. Según la opción elegida:
 - si es un piso, solicita planta y puerta;
 - si es una casa, solicita los metros de parcela.
6. El sistema crea el objeto correspondiente.
7. La vivienda queda almacenada en el inventario.

Resultado esperado

La vivienda se registra correctamente y pasa a formar parte de la colección interna del sistema.

Observaciones

Este caso de uso representa uno de los núcleos principales del programa y demuestra la correcta aplicación de herencia, especialización y validación de entrada.

19.4 Caso de uso: Listar inventario

Nombre del caso de uso

Mostrar el inventario actual de viviendas

Actor

Gestor inmobiliario

Descripción

Permite consultar todas las viviendas registradas hasta el momento, mostrando tanto su información común como sus datos específicos según el tipo de inmueble.

Flujo principal

1. El usuario accede a la opción "Listar Inventario".
2. El sistema recorre la colección de viviendas almacenadas.
3. Para cada vivienda:
 - muestra dirección y precio base;
 - detecta si se trata de un piso o una casa;
 - muestra la información específica correspondiente.
4. El sistema presenta el listado completo por consola.

Resultado esperado

El usuario obtiene una visión general del inventario actual de inmuebles.

Observaciones

Este caso de uso pone en práctica el uso de polimorfismo, `instanceof` y `casting` seguro.

19.5 Caso de uso: Registrar pago

Nombre del caso de uso

Registrar un pago mensual sobre una vivienda

Actor

Gestor inmobiliario

Descripción

Permite seleccionar una vivienda con saldo pendiente y registrar un abono económico sobre ella.

Flujo principal

1. El usuario accede a la opción “Registrar Pago”.
2. El sistema comprueba si existen viviendas registradas.
3. El sistema filtra aquellas viviendas que aún no están pagadas completamente.
4. El usuario selecciona una vivienda del listado.
5. El sistema solicita la cantidad a abonar.
6. El pago se registra sobre la vivienda seleccionada.
7. El sistema informa del nuevo estado de pago.

Resultado esperado

La vivienda actualiza su saldo pagado y su deuda pendiente.

Observaciones

Este caso de uso representa una de las primeras operaciones de gestión económica real del sistema.

19.6 Caso de uso: Aplicar subida anual

Nombre del caso de uso

Actualizar el precio de todas las viviendas

Actor

Gestor inmobiliario

Descripción

Permite aplicar un incremento porcentual al precio base de todas las viviendas registradas.

Flujo principal

1. El usuario accede a la opción “Aplicar IPC”.
2. El sistema solicita el porcentaje de subida.
3. El sistema recorre todas las viviendas almacenadas.
4. Se actualiza el precio base de cada una.
5. Se recalcula automáticamente la mensualidad con impuestos.

Resultado esperado

Todas las viviendas actualizan su precio base y su importe mensual total.

Observaciones

Este caso de uso añade una funcionalidad masiva de gestión que aporta realismo administrativo al sistema.

20. PRUEBAS REALIZADAS

20.1 Objetivo de las pruebas

Toda aplicación informática necesita ser sometida a pruebas para verificar que su comportamiento coincide con el esperado y que no falla ante situaciones normales o incorrectas de uso.

En este proyecto, las pruebas realizadas han tenido como objetivo principal comprobar:

- el correcto funcionamiento del flujo por consola,
- la estabilidad del sistema ante entradas inválidas,
- la creación adecuada de objetos,
- y la consistencia de la lógica económica implementada.

20.2 Pruebas del sistema de registro de viviendas

Se ha comprobado que el sistema permite registrar correctamente tanto viviendas de tipo **Piso** como viviendas de tipo **Casa**.

Aspectos verificados:

- introducción de dirección
- introducción de precio base
- diferenciación correcta entre piso y casa
- solicitud de atributos específicos según el tipo
- creación del objeto adecuado
- almacenamiento en la colección interna

Resultado obtenido:

La funcionalidad responde correctamente y registra las viviendas conforme al diseño previsto.

20.3 Pruebas del sistema de listado

Se ha verificado que el sistema es capaz de recorrer la colección de viviendas y mostrar correctamente tanto los datos comunes como los específicos.

Aspectos verificados:

- recorrido completo del inventario
- impresión de datos generales
- detección correcta del tipo de vivienda
- acceso a atributos específicos mediante casting

Resultado obtenido:

El sistema muestra la información esperada de forma clara y coherente.

20.4 Pruebas del sistema de pagos

Se han realizado pruebas orientadas a comprobar la lógica de pagos implementada dentro de la clase **Vivienda** y su integración en el menú principal.

Aspectos verificados:

- detección de viviendas pendientes de pago
- selección correcta de una vivienda desde el listado
- registro de pagos parciales
- registro de pagos completos
- actualización de deuda pendiente
- verificación de pago completo

Resultado obtenido:

La funcionalidad se comporta correctamente y permite una gestión básica del cobro mensual.

20.5 Pruebas del sistema de subida anual

Se ha comprobado el funcionamiento de la funcionalidad de actualización de precios mediante porcentaje.

Aspectos verificados:

- lectura correcta del porcentaje
- aplicación de la subida a todas las viviendas
- actualización del precio base
- recálculo de la mensualidad con impuestos

Resultado obtenido:

El sistema aplica correctamente la subida económica de forma masiva sobre todas las viviendas registradas.

20.6 Pruebas de comportamiento con listas vacías

Se han realizado pruebas específicas para comprobar el comportamiento del sistema cuando no existen viviendas registradas.

Situaciones probadas:

- listar inventario sin viviendas
- registrar pago sin viviendas
- aplicar subida anual sin viviendas

Resultado obtenido:

El sistema informa correctamente al usuario y evita operaciones inválidas sobre colecciones vacías.

21. CONTROL DE ERRORES Y ROBUSTEZ

21.1 Importancia del control de errores

Uno de los aspectos más importantes del proyecto ha sido la implementación de mecanismos que impidan que el programa se cierre de forma inesperada cuando el usuario introduce datos incorrectos.

Este punto es especialmente importante en una aplicación por consola, donde el sistema depende directamente de lo que escriba el usuario.

21.2 Estrategia general adoptada

El sistema se ha blindado mediante una combinación de:

- estructuras repetitivas `while`
- bloques `try-catch`
- validación de rangos válidos
- limpieza del buffer de entrada con `nextLine()`

Esta estrategia se aplica en prácticamente todos los puntos de interacción del sistema.

21.3 Tipos de errores contemplados

Entre los errores de entrada controlados actualmente se encuentran:

- introducción de letras en campos numéricos
- selección de opciones fuera del rango permitido
- valores no válidos en el menú
- errores de formato al introducir importes o porcentajes
- selección incorrecta del tipo de vivienda

Gracias a este enfoque, el sistema ofrece un comportamiento mucho más robusto y estable.

21.4 Valor técnico de esta parte del proyecto

Desde una perspectiva de calidad del software, el control de errores implementado representa uno de los puntos más sólidos del sistema actual.

Aunque visualmente pueda parecer menos llamativo que otras funcionalidades, en la práctica esta parte es fundamental para que el software sea usable, seguro y mantenible.

22. LIMITACIONES ACTUALES DE LA BETA

22.1 Introducción

La versión actual del proyecto ha sido concebida como una **BETA funcional**, es decir, una versión inicial operativa cuyo objetivo principal es consolidar la base estructural, lógica y técnica del sistema.

Por ello, existen varias limitaciones que no deben entenderse como fallos de diseño, sino como elementos conscientemente aplazados para futuras iteraciones.

22.2 Limitaciones técnicas actuales

Entre las principales limitaciones del sistema actual se encuentran:

- no existe persistencia permanente de datos,
- los inmuebles se pierden al cerrar el programa,
- no existe conexión con una base de datos,
- no hay diferenciación real entre usuarios,
- no existe autenticación,
- no se generan informes ni contratos,
- y no existe interfaz gráfica.

Estas limitaciones son completamente normales dentro del estado actual del desarrollo.

22.3 Limitaciones funcionales

Además de las limitaciones técnicas, el sistema todavía no incluye algunas funcionalidades de negocio que ya están previstas en la hoja de ruta del proyecto, como por ejemplo:

- historial completo de operaciones por usuario,
- control avanzado de rentabilidad,
- beneficios por propietario,
- contratos de alquiler,
- gestión de incidencias,
- exportación documental,
- o paneles administrativos más completos.

23. VISIÓN EVOLUTIVA DEL PROYECTO

23.1 Filosofía de evolución

El proyecto no ha sido concebido como una aplicación cerrada desde el principio, sino como una solución evolutiva que irá creciendo por fases.

Esta decisión es muy importante desde el punto de vista del desarrollo de software, ya que permite:

- construir una base sólida,
- validar primero la estructura,
- y ampliar después el sistema sin comprometer su estabilidad.

23.2 Diferenciación entre BETA y versión oficial

La fase **BETA** actual representa el núcleo inicial del proyecto, centrado en:

- gestión de inmuebles,
- control básico de pagos,
- cálculo de impuestos canarios,
- y actualización económica simple.

La futura **versión 1.0 oficial** supondrá un salto cualitativo importante, incorporando nuevas capas funcionales, mayor organización interna y una visión más cercana a un software real de gestión inmobiliaria.

24. ROADMAP DEL PROYECTO

24.1 Fase actual: BETA

La versión BETA del sistema se centra en consolidar el núcleo del programa.

Elementos incluidos en esta fase:

- clase base Vivienda
- subclases Piso y Casa
- menú principal por consola
- registro de inmuebles
- listado del inventario
- registro de pagos
- subida porcentual anual
- cálculo con IGIC
- validación robusta de entradas

Objetivo de esta fase:

Construir una base estable, limpia y ampliable.

24.2 Fase siguiente: versión 1.0

La versión **1.0** deberá centrarse en consolidar el producto como un sistema más completo y mejor estructurado.

Mejoras previstas:

- reorganización interna del código
- refactorización del menú principal
- ampliación de la lógica financiera
- mejor separación de responsabilidades
- mayor preparación para crecimiento futuro

Objetivo de esta fase:

Transformar la BETA en una primera versión formal y mantenible del sistema.

24.3 Versión 1.1

La versión **1.1** estará orientada a la adaptación fiscal y funcional fuera del ámbito canario.

Mejoras previstas:

- incorporación del sistema de IVA
- soporte para otros impuestos aplicables en la península
- configuración fiscal adaptable según territorio

Objetivo de esta fase:

Expandir el proyecto más allá del contexto exclusivamente canario.

24.4 Versiones 1.2 a 1.5

Estas versiones estarán orientadas a la ampliación de perfiles y lógica de negocio.

Funcionalidades previstas:

- diferenciación entre usuario corriente e inversor/arrendador
- historial individual de acciones
- control de beneficios
- métricas de rentabilidad
- métodos como `getBeneficioTotal()`
- primeras bases para trazabilidad de operaciones

Objetivo de esta fase:

Convertir el sistema en una herramienta de gestión más realista y cercana a una plataforma inmobiliaria completa.

24.5 Fases posteriores

A medio y largo plazo, el proyecto podrá incorporar:

- base de datos
- persistencia real
- autenticación
- exportación documental
- interfaz gráfica
- panel de control
- automatización de contratos
- informes económicos
- integración multiusuario

Estas fases no forman parte del alcance inmediato, pero sí constituyen una dirección natural de crecimiento del sistema.

25. ADAPTACIÓN AL CONTEXTO CANARIO

25.1 Enfoque territorial del proyecto

Una de las decisiones más importantes del proyecto ha sido contextualizar la solución dentro del ámbito de **Canarias**, lo que le aporta coherencia práctica y realismo funcional.

En lugar de construir desde el principio un sistema genérico para toda España, se ha optado por diseñar una BETA alineada con las particularidades fiscales del territorio canario.

25.2 Uso del IGIC en lugar del IVA

La diferencia fiscal más importante incorporada al sistema ha sido la aplicación del **IGIC** en lugar del **IVA**.

Esto permite que la lógica económica del programa sea coherente con la realidad tributaria canaria, reforzando el valor contextual del proyecto.

Además, esta decisión demuestra capacidad de adaptación del software al entorno en el que se pretende aplicar.

25.3 Valor técnico de esta adaptación

Aunque pueda parecer un detalle pequeño, la incorporación del IGIC en lugar del IVA tiene un valor técnico importante porque muestra que el sistema no se ha diseñado de forma abstracta o genérica, sino teniendo en cuenta un contexto real de uso.

Esto aporta mayor credibilidad al proyecto y abre la puerta a futuras versiones configurables según territorio.

26. POSIBLES MEJORAS FUTURAS

26.1 Mejoras técnicas

Entre las mejoras técnicas más relevantes que podrían incorporarse en versiones futuras destacan:

- conexión con base de datos
- persistencia de información
- serialización o exportación de datos
- separación por capas
- refactorización hacia una arquitectura más limpia
- validaciones más centralizadas
- y mejor modularización del menú

26.2 Mejoras funcionales

Entre las mejoras funcionales más interesantes destacan:

- historial de operaciones por usuario
- cálculo de rentabilidad anual
- gestión de propietarios
- seguimiento de ingresos
- previsiones financieras
- exportación de contratos
- control de incidencias
- y estadísticas inmobiliarias

26.3 Mejoras de experiencia de usuario

A nivel de experiencia de uso, futuras versiones podrían incluir:

- interfaz gráfica
- navegación más intuitiva
- formularios guiados
- filtros de búsqueda
- confirmaciones visuales
- y paneles de resumen

Estas mejoras permitirían acercar el sistema a un entorno de uso más profesional y comercial.

27. CONCLUSIONES

El desarrollo de esta versión BETA del sistema de gestión inmobiliaria ha permitido construir una base sólida, funcional y técnicamente coherente sobre la que seguir evolucionando el proyecto.

A lo largo del desarrollo, se han aplicado conceptos fundamentales de programación como:

- herencia,
- encapsulamiento,
- polimorfismo,
- validación de entradas,
- gestión dinámica de objetos,
- y lógica de negocio orientada al dominio.

Además, el proyecto no se ha limitado a un simple ejercicio académico, sino que ya incorpora una orientación clara hacia una posible evolución real como software de gestión.

La implementación actual demuestra que se ha trabajado no solo en que el programa “funcione”, sino también en que esté construido con una lógica razonable, una estructura ampliable y una visión de producto bien definida.

Por tanto, puede concluirse que esta fase BETA cumple adecuadamente su propósito: servir como base técnica, conceptual y funcional para el desarrollo futuro de una versión más completa, robusta y profesional del sistema.

28. CIERRE DEL DOCUMENTO

La presente memoria recoge el análisis, diseño, implementación y visión evolutiva del proyecto de gestión inmobiliaria desarrollado en Java.

El documento refleja tanto el estado actual de la solución como la dirección prevista para sus futuras versiones, dejando constancia de una arquitectura inicial bien planteada y de un enfoque progresivo de desarrollo.

A partir de esta base, el proyecto se encuentra preparado para continuar su evolución hacia versiones más completas, con nuevas funcionalidades, nuevos perfiles de usuario y una mayor integración tecnológica